

Resolver os exercícios de listas/vetores/arrays em Python

Ex1.:

```
temperaturas = []
temperaturas.append(float(input("Digite a temperatura 1: ")))
temperaturas.append(float(input("Digite a temperatura 2: ")))
temperaturas.append(float(input("Digite a temperatura 3: ")))
temperaturas.append(float(input("Digite a temperatura 4: ")))
temperaturas.append(float(input("Digite a temperatura 5: ")))
temperaturas.append(float(input("Digite a temperatura 6: ")))
```

```
maximo = max(temperaturas)
```

```
minimo = min(temperaturas)
```

```
print("A temperatura máxima é: ", maximo)
```

```
print("A temperatura mínima é: ", minimo)
```



```
temperaturas = []
temperaturas.append(float(input("Digite a temperatura 1: ")))
temperaturas.append(float(input("Digite a temperatura 2: ")))
temperaturas.append(float(input("Digite a temperatura 3: ")))
temperaturas.append(float(input("Digite a temperatura 4: ")))
temperaturas.append(float(input("Digite a temperatura 5: ")))
temperaturas.append(float(input("Digite a temperatura 6: ")))
```

```
maximo = max(temperaturas)
```

```
minimo = min(temperaturas)
```

```
print("A temperatura máxima é: ", maximo)
```

```
print("A temperatura mínima é: ", minimo)
```

```
... Digite a temperatura 1: 820
    Digite a temperatura 2: 835
    Digite a temperatura 3: 848
    Digite a temperatura 4: 791
    Digite a temperatura 5: 812
    Digite a temperatura 6: 856
    A temperatura máxima é:  856.0
    A temperatura mínima é:  791.0
```

Ex2.:

```
tensao = []
```

```
tensao.append(float(input("Digite a tensão 1: ")))
```

```
tensao.append(float(input("Digite a tensão 2: ")))
```

```
tensao.append(float(input("Digite a tensão 3: ")))
```

```

tensao.append(float(input("Digite a tensão 4: ")))
tensao.append(float(input("Digite a tensão 5: ")))
tensao.append(float(input("Digite a tensão 6: ")))
tensao.append(float(input("Digite a tensão 7: ")))
tensao.append(float(input("Digite a tensão 8: ")))

```

```

media = sum(tensao) / len(tensao)
print("A média das tensões é: ", media)

```

```

for e in tensao:
    if e < 10.8 or e > 13.2:
        print (f"Tensão {e}: FORA DA FAIXA!")
    else:
        print (f"Tensão {e}: DENTRO DA FAIXA!")

```

```

❏ tensao = []
tensao.append(float(input("Digite a tensão 1: ")))
tensao.append(float(input("Digite a tensão 2: ")))
tensao.append(float(input("Digite a tensão 3: ")))
tensao.append(float(input("Digite a tensão 4: ")))
tensao.append(float(input("Digite a tensão 5: ")))
tensao.append(float(input("Digite a tensão 6: ")))
tensao.append(float(input("Digite a tensão 7: ")))
tensao.append(float(input("Digite a tensão 8: ")))

media = sum(tensao) / len(tensao)
print("A média das tensões é: ", media)

for e in tensao:
    if e < 10.8 or e > 13.2:
        print (f"Tensão {e}: FORA DA FAIXA!")
    else:
        print (f"Tensão {e}: DENTRO DA FAIXA!")

```

```

... Digite a tensão 1: 11.9
    Digite a tensão 2: 12.1
    Digite a tensão 3: 12.0
    Digite a tensão 4: 13.5
    Digite a tensão 5: 11.7
    Digite a tensão 6: 12.3
    Digite a tensão 7: 10.5
    Digite a tensão 8: 12.0
    A média das tensões é: 12.0
    Tensão 11.9: DENTRO DA FAIXA!
    Tensão 12.1: DENTRO DA FAIXA!
    Tensão 12.0: DENTRO DA FAIXA!
    Tensão 13.5: FORA DA FAIXA!
    Tensão 11.7: DENTRO DA FAIXA!
    Tensão 12.3: DENTRO DA FAIXA!
    Tensão 10.5: FORA DA FAIXA!
    Tensão 12.0: DENTRO DA FAIXA!

```

Ex3.: Não entendi

Ex4.:

```

vibracao = [0.12, 0.34, -999.0, 0.28, 0.91, -999.0, 1.42, 0.67]

```

```
while -999.0 in vibracao:
    vibracao.remove(-999.0)

print(vibracao)
print("Quantidade: ",len(vibracao))
```

🔴 `vibracao = [0.12, 0.34, -999.0, 0.28, 0.91, -999.0, 1.42, 0.67]`

```
while -999.0 in vibracao:
    vibracao.remove(-999.0)

print(vibracao)
print("Quantidade: ",len(vibracao))
```

```
... [0.12, 0.34, 0.28, 0.91, 1.42, 0.67]
    Quantidade:  6
```

Ex5.:

```
cargas = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]
```

```
initial_total_carga = sum(cargas)
```

```
items_to_remove = [8.0, 40.0]
```

```
cargas_ativas = [carga for carga in cargas if carga not in
items_to_remove]
```

```
final_total_carga = sum(cargas_ativas)
```

```
reduction = initial_total_carga - final_total_carga
```

```
print("Cargas Ativas: ", cargas_ativas)
```

```
print(f"Potencia antes: {initial_total_carga} kW")
```

```
print(f"Potencia agora: {final_total_carga} kW")
```

```
print(f"Redução: {reduction} kW")
```

```

▶ cargas = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]

initial_total_carga = sum(cargas)

items_to_remove = [8.0, 40.0]
cargas_ativas = [carga for carga in cargas if carga not in items_to_remove]

final_total_carga = sum(cargas_ativas)
reduction = initial_total_carga - final_total_carga

print("Cargas Ativas: ", cargas_ativas)
print(f"Potencia antes: {initial_total_carga} kW")
print(f"Potencia agora: {final_total_carga} kW")
print(f"Redução: {reduction} kW")

```

```

*** Cargas Ativas: [15.0, 22.5, 12.5, 30.0]
Potencia antes: 128.0 kW
Potencia agora: 80.0 kW
Redução: 48.0 kW

```

Ex6.:

```

canais_original = [850, 900, 950, 1800, 2100, 2600]
interferencia_original = [-65, -80, -72, -55, -90, -68]

limiar_corte = -70

canais_ativos = []

canal_interferencia_map = dict(zip(canais_original,
interferencia_original))

for canal, interferencia_valor in canal_interferencia_map.items():
    if interferencia_valor > limiar_corte:
        print(f"Canal {canal} MHz desativado (interferência:
{interferencia_valor} dBm)")
    else:
        canais_ativos.append(canal)

canais_ativos.sort()

print(f"Canais ativos: {canais_ativos}")

```

```

▶ canais_original = [850, 900, 950, 1800, 2100, 2600]
  interferencia_original = [-65, -80, -72, -55, -90, -68]

  limiar_corte = -70
  canais_ativos = []

  canal_interferencia_map = dict(zip(canais_original, interferencia_original))

  for canal, interferencia_valor in canal_interferencia_map.items():
      if interferencia_valor > limiar_corte:
          print(f"Canal {canal} MHz desativado (interferência: {interferencia_valor} dBm)")
      else:
          canais_ativos.append(canal)
  canais_ativos.sort()

  print(f"Canais ativos: {canais_ativos}")

*** Canal 850 MHz desativado (interferência: -65 dBm)
    Canal 1800 MHz desativado (interferência: -55 dBm)
    Canal 2600 MHz desativado (interferência: -68 dBm)
    Canais ativos: [900, 950, 2100]

```

Ex7.:

```

params_originais = [120.0, 150.0, 95.0, 200.0, 110.0]

params_backup = params_originais.copy()
params_originais[0] = 130.0

print(f"Original (modificada): {params_originais}")
print(f"Backup (preservado): {params_backup}")

print("\n--- Demonstrando o que aconteceria se usasse = em vez de
copy() ---")

params_originais_para_demo = [120.0, 150.0, 95.0, 200.0, 110.0]

params_referencia = params_originais_para_demo
params_originais_para_demo[0] = 130.0

print(f"Original (modificada): {params_originais_para_demo}")
print(f"Referência (também modificada): {params_referencia}")

```

```

▶ params_originais = [120.0, 150.0, 95.0, 200.0, 110.0]

params_backup = params_originais.copy()
params_originais[0] = 130.0

print(f"Original (modificada): {params_originais}")
print(f"Backup (preservado): {params_backup}")

print("\n--- Demonstrando o que aconteceria se usasse = em vez de copy() ---")

params_originais_para_demo = [120.0, 150.0, 95.0, 200.0, 110.0]

params_referencia = params_originais_para_demo
params_originais_para_demo[0] = 130.0

print(f"Original (modificada): {params_originais_para_demo}")
print(f"Referência (também modificada): {params_referencia}")

```

```

*** Original (modificada): [130.0, 150.0, 95.0, 200.0, 110.0]
    Backup (preservado): [120.0, 150.0, 95.0, 200.0, 110.0]

    --- Demonstrando o que aconteceria se usasse = em vez de copy() ---
    Original (modificada): [130.0, 150.0, 95.0, 200.0, 110.0]
    Referência (também modificada): [130.0, 150.0, 95.0, 200.0, 110.0]

```

Ex8.:

```

nominal = [47.0, 100.0, 220.0, 33.0, 68.0]

simulacao = nominal.copy()

resistor_em_curto = 100.0
indice_resistor = simulacao.index(resistor_em_curto)
simulacao[indice_resistor] = 0.0

req_nominal = sum(nominal)
req_falha = sum(simulacao)
variacao = req_nominal - req_falha

print(f"Nominal      : {nominal}")
print(f"Simulação     : {simulacao}")
print(f"Req nominal    : {req_nominal} Ω")
print(f"Req falha      : {req_falha} Ω")
print(f"Variação       : {variacao} Ω")

```

```

▶ nominal = [47.0, 100.0, 220.0, 33.0, 68.0]

simulacao = nominal.copy()

resistor_em_curto = 100.0
indice_resistor = simulacao.index(resistor_em_curto)
simulacao[indice_resistor] = 0.0

req_nominal = sum(nominal)
req_falha = sum(simulacao)
variacao = req_nominal - req_falha

print(f"Nominal      : {nominal}")
print(f"Simulação     : {simulacao}")
print(f"Req nominal    : {req_nominal} Ω")
print(f"Req falha      : {req_falha} Ω")
print(f"Variação       : {variacao} Ω")

```

```

*** Nominal      : [47.0, 100.0, 220.0, 33.0, 68.0]
    Simulação     : [47.0, 0.0, 220.0, 33.0, 68.0]
    Req nominal   : 468.0 Ω
    Req falha     : 368.0 Ω
    Variação      : 100.0 Ω

```

Ex9.:

```

tabela_atual = ['10.0.0.0/8', '172.16.0.0/12', '192.168.1.0/24',
                '203.0.113.0/24', '198.51.100.0/24']
rota_removida = '203.0.113.0/24'
rotas_inseridas = ['10.10.0.0/16', '192.168.100.0/24']

tabela_antiga = tabela_atual.copy()
tabela_nova = tabela_atual.copy()
tabela_nova.remove(rota_removida)
tabela_nova.extend(rotas_inseridas)

rotas_adicionadas = [r for r in tabela_nova if r not in tabela_antiga]
rotas_removidas = [r for r in tabela_antiga if r not in tabela_nova]

print(f"Rotas adicionadas: {rotas_adicionadas}")
print(f"Rotas removidas : {rotas_removidas}")

```

```

▶ tabela_atual = ['10.0.0.0/8', '172.16.0.0/12', '192.168.1.0/24',
                  '203.0.113.0/24', '198.51.100.0/24']
rota_removida = '203.0.113.0/24'
rotas_inseridas = ['10.10.0.0/16', '192.168.100.0/24']

tabela_antiga = tabela_atual.copy()
tabela_nova = tabela_atual.copy()
tabela_nova.remove(rota_removida)
tabela_nova.extend(rotas_inseridas)

rotas_adicionadas = [r for r in tabela_nova if r not in tabela_antiga]
rotas_removidas = [r for r in tabela_antiga if r not in tabela_nova]

print(f"Rotas adicionadas: {rotas_adicionadas}")
print(f"Rotas removidas : {rotas_removidas}")

```

```

*** Rotas adicionadas: ['10.10.0.0/16', '192.168.100.0/24']
    Rotas removidas : ['203.0.113.0/24']

```

Ex10.:

```

sensor_A = [3.2, 3.4, 3.1, 3.5]
sensor_B = [4.1, 4.0, 4.3, 3.9]
sensor_C = [2.8, 2.9, 3.0, 2.7]

todas_leituras = []
todas_leituras.extend(sensor_A)
todas_leituras.extend(sensor_B)
todas_leituras.extend(sensor_C)

total_leituras = len(todas_leituras)
pressao_media = sum(todas_leituras) / total_leituras
pressao_maxima = max(todas_leituras)
pressao_minima = min(todas_leituras)

print(f"Total de leituras: {total_leituras}")
print(f"Pressão média      : {pressao_media:.2f} bar")
print(f"Pressão máxima     : {pressao_maxima} bar")
print(f"Pressão mínima      : {pressao_minima} bar")

```



```

▶ sensor_A = [3.2, 3.4, 3.1, 3.5]
  sensor_B = [4.1, 4.0, 4.3, 3.9]
  sensor_C = [2.8, 2.9, 3.0, 2.7]

todas_leituras = []
todas_leituras.extend(sensor_A)
todas_leituras.extend(sensor_B)
todas_leituras.extend(sensor_C)

total_leituras = len(todas_leituras)
pressao_media = sum(todas_leituras) / total_leituras
pressao_maxima = max(todas_leituras)
pressao_minima = min(todas_leituras)

print(f"Total de leituras: {total_leituras}")
print(f"Pressão média : {pressao_media:.2f} bar")
print(f"Pressão máxima : {pressao_maxima} bar")
print(f"Pressão mínima : {pressao_minima} bar")

```

```

... Total de leituras: 12
  Pressão média : 3.41 bar
  Pressão máxima : 4.3 bar
  Pressão mínima : 2.7 bar

```

Ex11.:

```

medidor_1 = [48.2, 51.0, 49.7, 47.5, 52.3]
medidor_2 = [50.1, 46.8, 53.0, 49.0, 55.2]

todas_medicoes = medidor_1.copy()
todas_medicoes.extend(medidor_2)
todas_medicoes.sort()
media = sum(todas_medicoes) / len(todas_medicoes)

tolerancia_percentual = 0.15
tolerancia_valor = media * tolerancia_percentual

limite_inferior = media - tolerancia_valor
limite_superior = media + tolerancia_valor
valores_fora_faixa = [m for m in todas_medicoes if m < limite_inferior
or m > limite_superior]

print(f"Medições ordenadas: {todas_medicoes}")
print(f"Média: {media:.2f} Ω | Tolerância: ±{tolerancia_valor:.2f} Ω")

if valores_fora_faixa:

```

```

    print(f"Valores fora da faixa de tolerância: {valores_fora_faixa}")
else:
    print("(nenhum valor fora da faixa neste conjunto)")

```

```

▶ medidor_1 = [48.2, 51.0, 49.7, 47.5, 52.3]
  medidor_2 = [50.1, 46.8, 53.0, 49.0, 55.2]

todas_medicoes = medidor_1.copy()
todas_medicoes.extend(medidor_2)
todas_medicoes.sort()
media = sum(todas_medicoes) / len(todas_medicoes)

tolerancia_percentual = 0.15
tolerancia_valor = media * tolerancia_percentual

limite_inferior = media - tolerancia_valor
limite_superior = media + tolerancia_valor
valores_fora_faixa = [m for m in todas_medicoes if m < limite_inferior or m > limite_superior]

print(f"Medições ordenadas: {todas_medicoes}")
print(f"Média: {media:.2f} Ω | Tolerância: ±{tolerancia_valor:.2f} Ω")

if valores_fora_faixa:
    print(f"Valores fora da faixa de tolerância: {valores_fora_faixa}")
else:
    print("(nenhum valor fora da faixa neste conjunto)")

```

```

*** Medições ordenadas: [46.8, 47.5, 48.2, 49.0, 49.7, 50.1, 51.0, 52.3, 53.0, 55.2]
    Média: 50.28 Ω | Tolerância: ±7.54 Ω
    (nenhum valor fora da faixa neste conjunto)

```

Ex12.:

```

antena_norte = [-85, -90, -105, -78, -92, -110, -88]
antena_sul   = [-95, -88, -102, -79, -85, -91, -97]
antena_leste = [-80, -75, -108, -93, -86, -103, -82]
limiar_cobertura = -100 # dBm

```

```

todas_leituras = []
todas_leituras.extend(antena_norte)
todas_leituras.extend(antena_sul)
todas_leituras.extend(antena_leste)

total_amostras = len(todas_leituras)

```

```

leituras_em_cobertura = [leitura for leitura in todas_leituras if
leitura >= limiar_cobertura]
quantidade_em_cobertura = len(leituras_em_cobertura)
taxa_cobertura = (quantidade_em_cobertura / total_amostras) * 100

pior_sinal = min(todas_leituras)
indice_pior_sinal = todas_leituras.index(pior_sinal)
media_leituras = sum(todas_leituras) / total_amostras

print(f"Total de amostras      : {total_amostras}")
print(f"Em cobertura (>={limiar_cobertura}):
{quantidade_em_cobertura}")
print(f"Taxa de cobertura       : {taxa_cobertura:.1f}%")
print(f"Pior sinal             : {pior_sinal} dBm (índice
{indice_pior_sinal})")
print(f"Média                  : {media_leituras:.1f} dBm")

▶ antena_norte = [-85, -90, -105, -78, -92, -110, -88]
  antena_sul   = [-95, -88, -102, -79, -85, -91, -97]
  antena_leste = [-80, -75, -108, -93, -86, -103, -82]
  limiar_cobertura = -100 # dBm

todas_leituras = []
todas_leituras.extend(antena_norte)
todas_leituras.extend(antena_sul)
todas_leituras.extend(antena_leste)

total_amostras = len(todas_leituras)
leituras_em_cobertura = [leitura for leitura in todas_leituras if leitura >= limiar_cobertura]
quantidade_em_cobertura = len(leituras_em_cobertura)
taxa_cobertura = (quantidade_em_cobertura / total_amostras) * 100

pior_sinal = min(todas_leituras)
indice_pior_sinal = todas_leituras.index(pior_sinal)
media_leituras = sum(todas_leituras) / total_amostras

print(f"Total de amostras      : {total_amostras}")
print(f"Em cobertura (>={limiar_cobertura}): {quantidade_em_cobertura}")
print(f"Taxa de cobertura       : {taxa_cobertura:.1f}%")
print(f"Pior sinal             : {pior_sinal} dBm (índice {indice_pior_sinal})")
print(f"Média                  : {media_leituras:.1f} dBm")

*** Total de amostras      : 21
    Em cobertura (>=-100): 16
    Taxa de cobertura      : 76.2%
    Pior sinal             : -110 dBm (índice 5)
    Média                  : -91.0 dBm

```

Ex13.:

```
medicoes = [50.02, 49.97, 50.06, 49.94, 50.00,
```

```

        50.08, 49.99, 50.01, 49.92, 50.04]
diametro_nominal = 50.00
tolerancia      = 0.05

aprovadas = []
reprovadas = []

for medida in medicoes:
    if abs(medida - diametro_nominal) <= tolerancia:
        aprovadas.append(medida)
    else:
        reprovadas.append(medida)

indice_aprovacao = (len(aprovadas) / len(medicoes)) * 100

print(f"Aprovadas ({len(aprovadas)}): {aprovadas}")
print(f"Reprovadas ({len(reprovadas)}): {reprovadas}")
print(f"Índice de aprovação: {indice_aprovacao:.0f}%")

```

```

▶ medicoes = [50.02, 49.97, 50.06, 49.94, 50.00,
              50.08, 49.99, 50.01, 49.92, 50.04]
diametro_nominal = 50.00
tolerancia      = 0.05

aprovadas = []
reprovadas = []

for medida in medicoes:
    if abs(medida - diametro_nominal) <= tolerancia:
        aprovadas.append(medida)
    else:
        reprovadas.append(medida)

indice_aprovacao = (len(aprovadas) / len(medicoes)) * 100

print(f"Aprovadas ({len(aprovadas)}): {aprovadas}")
print(f"Reprovadas ({len(reprovadas)}): {reprovadas}")
print(f"Índice de aprovação: {indice_aprovacao:.0f}%")

```

```

... Aprovadas (6): [50.02, 49.97, 50.0, 49.99, 50.01, 50.04]
   Reprovadas (4): [50.06, 49.94, 50.08, 49.92]
   Índice de aprovação: 60%

```

Ex14.:

```
cenario_atual = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]
```

```

novas_cargas = [55.0, 18.0, 75.0]

total_atual = sum(cenario_atual)
media_atual = total_atual / len(cenario_atual)
maior_atual = max(cenario_atual)

cenario_futuro = cenario_atual.copy()
cenario_futuro.extend(novas_cargas)

total_futuro = sum(cenario_futuro)
media_futuro = total_futuro / len(cenario_futuro)
maior_futuro = max(cenario_futuro)

print("---- Cenário atual ----")
print(f" Total : {total_atual:.1f} kW | Média : {media_atual:.1f} kW  

| Maior : {maior_atual:.1f} kW")
print("---- Cenário futuro ----")
print(f" Total : {total_futuro:.1f} kW | Média : {media_futuro:.1f}  

kW | Maior : {maior_futuro:.1f} kW")

```

```

▶ cenario_atual = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]
novas_cargas = [55.0, 18.0, 75.0]

total_atual = sum(cenario_atual)
media_atual = total_atual / len(cenario_atual)
maior_atual = max(cenario_atual)

cenario_futuro = cenario_atual.copy()
cenario_futuro.extend(novas_cargas)

total_futuro = sum(cenario_futuro)
media_futuro = total_futuro / len(cenario_futuro)
maior_futuro = max(cenario_futuro)

print("--- Cenário atual ---")
print(f" Total : {total_atual:.1f} kW | Média : {media_atual:.1f} kW | Maior : {maior_atual:.1f} kW")
print("--- Cenário futuro ---")
print(f" Total : {total_futuro:.1f} kW | Média : {media_futuro:.1f} kW | Maior : {maior_futuro:.1f} kW")

```

```

*** --- Cenário atual ---
    Total : 128.0 kW | Média : 21.3 kW | Maior : 40.0 kW
--- Cenário futuro ---
    Total : 276.0 kW | Média : 30.7 kW | Maior : 75.0 kW

```

Ex15.:

```
import math
```

```

canal_I = [0.82, 0.79, 0.85, 5.20, 0.81, 0.78, 0.83]
canal_Q = [0.41, 0.39, 0.43, 0.40, -4.80, 0.42, 0.38]

```

```

sinal_bruto = []
sinal_bruto.extend(canal_I)
sinal_bruto.extend(canal_Q)

sinal_backup = sinal_bruto.copy()

total_amostras = len(sinal_bruto)
media = sum(sinal_bruto) / total_amostras

soma_quadrados_diferencas = sum([(x - media)**2 for x in sinal_bruto])
desvio_padrao = math.sqrt(soma_quadrados_diferencas / total_amostras)

limite_inferior = media - (3 * desvio_padrao)
limite_superior = media + (3 * desvio_padrao)

ruidos_encontrados_para_remocao = []
for valor in sinal_bruto:
    if not (limite_inferior <= valor <= limite_superior):
        ruidos_encontrados_para_remocao.append(valor)

sinal_limpo_final = sinal_bruto.copy()
for ruído in ruidos_encontrados_para_remocao:
    if ruído in sinal_limpo_final:
        sinal_limpo_final.remove(ruído)

ruidos_removidos = ruidos_encontrados_para_remocao

print(f"Sinal bruto ({len(sinal_bruto)} amostras): {sinal_bruto}")
print(f"Ruídos removidos : {ruidos_removidos}")
print(f"Sinal limpo ({len(sinal_limpo_final)} amostras): {sinal_limpo_final}")

```

```

import math

canal_I = [0.82, 0.79, 0.85, 5.20, 0.81, 0.78, 0.83]
canal_Q = [0.41, 0.39, 0.43, 0.40, -4.80, 0.42, 0.38]

sinal_bruto = []
sinal_bruto.extend(canal_I)
sinal_bruto.extend(canal_Q)

sinal_backup = sinal_bruto.copy()

total_amostras = len(sinal_bruto)
media = sum(sinal_bruto) / total_amostras

soma_quadrados_diferencas = sum([(x - media)**2 for x in sinal_bruto])
desvio_padrao = math.sqrt(soma_quadrados_diferencas / total_amostras)

limite_inferior = media - (3 * desvio_padrao)
limite_superior = media + (3 * desvio_padrao)

ruidos_encontrados_para_remocao = []
for valor in sinal_bruto:
    if not (limite_inferior <= valor <= limite_superior):
        ruidos_encontrados_para_remocao.append(valor)

sinal_limpo_final = sinal_bruto.copy()
for ruído in ruidos_encontrados_para_remocao:
    if ruído in sinal_limpo_final:
        sinal_limpo_final.remove(ruído)

ruidos_removidos = ruidos_encontrados_para_remocao

print(f"Sinal bruto ({len(sinal_bruto)} amostras): {sinal_bruto}")
print(f"Ruídos removidos : {ruidos_removidos}")
print(f"Sinal limpo ({len(sinal_limpo_final)} amostras): {sinal_limpo_final}")

*** Sinal bruto (14 amostras): [0.82, 0.79, 0.85, 5.2, 0.81, 0.78, 0.83, 0.41, 0.39, 0.43, 0.4, -4.8, 0.42, 0.38]
Ruídos removidos : []
Sinal limpo (14 amostras): [0.82, 0.79, 0.85, 5.2, 0.81, 0.78, 0.83, 0.41, 0.39, 0.43, 0.4, -4.8, 0.42, 0.38]

```